

Lecture 6: Parallel programming

Threads, channels and shared state

Willem Vanhulle

DevLab Rust 2025

Tuesday December 9, 2025

github.com/wvhulle/rust-course-ghent

Make sure you have already finished:

- `session-5/tests/error-handling.rs`

1. Threads	2
1.1. Plain Threads	3
1.2. Receiving data from threads	5
1.3. Handling thread panics	6
1.4. Sending references to threads	7
1.5. Borrowing and threads (continued)	8
1.6. Scoped threads	9
1.7. Multiple scoped threads	10
1.8. Moving to threads	11
1.9. Review: what is 'static'?	12
2. Channels	13
3. Send and Sync	18
4. Shared state	29
5. Mutexes	33

1.1. Plain Threads

```
1  use std::thread;
2  use std::time::Duration;
3
4  fn main() {
5      thread::spawn(|| {
6          for i in 0..10 {
7              println!("Count in thread: {i}!");
8              thread::sleep(Duration::from_millis(5));
9          }
10     });
11
12     for i in 0..5 {
13         println!("Main thread: {i}");
14         thread::sleep(Duration::from_millis(5));
15     }
16 }
```

(playground link)

What happens to the spawned thread?

1.1. Plain Threads

```
1  use std::thread;
2  use std::time::Duration;
3
4  fn main() {
5      thread::spawn(|| {
6          for i in 0..10 {
7              println!("Count in thread: {i}!");
8              thread::sleep(Duration::from_millis(5));
9          }
10     });
11
12     for i in 0..5 {
13         println!("Main thread: {i}");
14         thread::sleep(Duration::from_millis(5));
15     }
16 }
```

(playground link)

What happens to the spawned thread?

Terminated by operating system when main thread exits. No destructors called.

1.1. Plain Threads

```
1 thread::spawn(|| {  
2     for i in 0..10 {  
3     });  
4  
5 for i in 0..5 {  
6     println!("Main thread: {i}");  
7     thread::sleep(Duration::from_millis(5));  
8 }
```

([playground link](#))

How can we ensure the spawned thread finishes?

1.1. Plain Threads

```
1 thread::spawn(|| {  
2     for i in 0..10 {  
3     });  
4  
5 for i in 0..5 {  
6     println!("Main thread: {i}");  
7     thread::sleep(Duration::from_millis(5));  
8 }
```

[\(playground link\)](#)

How can we ensure the spawned thread finishes?

Use `JoinHandle` returned by `thread::spawn` and call `join()` on it.

```
1 fn main() {  
2     let handle = thread::spawn(|| {  
3         // thread code  
4     });  
5     // main thread code  
6     handle.join();  
7 }
```

[\(playground link\)](#)

1.2. Receiving data from threads

How can we get a return value from a thread?

1.2. Receiving data from threads

How can we get a return value from a thread?

The `JoinHandle` returns a `Result` with the thread's return value.

```
1 fn main() {  
2     let handle = thread::spawn(|| {  
3         42  
4     });  
5  
6     let result = handle.join().unwrap();  
7     println!("The answer is: {}", result);  
8 }
```

(playground link)

What happens to the main thread if a spawned thread panics?

1.2. Receiving data from threads

How can we get a return value from a thread?

The `JoinHandle` returns a `Result` with the thread's return value.

```
1 fn main() {  
2     let handle = thread::spawn(|| {  
3         42  
4     });  
5  
6     let result = handle.join().unwrap();  
7     println!("The answer is: {}", result);  
8 }
```

(playground link)

What happens to the main thread if a spawned thread panics?

Nothing, the main thread continues executing.

1.3. Handling thread panics

If we don't join, we never see the panic.

1.3. Handling thread panics

If we don't join, we never see the panic.

If we don't handle the `Result` from `join()`, the panic is also ignored.

1.3. Handling thread panics

If we don't join, we never see the panic.

If we don't handle the Result from join(), the panic is also ignored.

We need to check the Result from join() to see if the thread panicked:

```
1  fn main() {  
2      let handle = thread::spawn(|| {  
3          panic!("Thread panicked!");  
4      });  
5      let result = handle.join();  
6      match result {  
7          Ok(_) => println!("Thread completed successfully."),  
8          Err(e) => println!("Thread panicked: {:?}", e),  
9      }  
10 }
```

(playground link)

Warning

You are responsible for handling panics in spawned threads! Rust will not do this for you.

Exercise:

- examples/join-handle.rs

1.4. Sending references to threads

Not all references can be sent to threads.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         let sum: i32 = data.iter().sum();  
5         println!("Sum: {sum}");  
6     });  
7     data.push(4); // modify while thread may be reading  
8     handle.join().unwrap();  
9 }
```

(playground link)

What happens if a thread borrows data from the main thread?

1.4. Sending references to threads

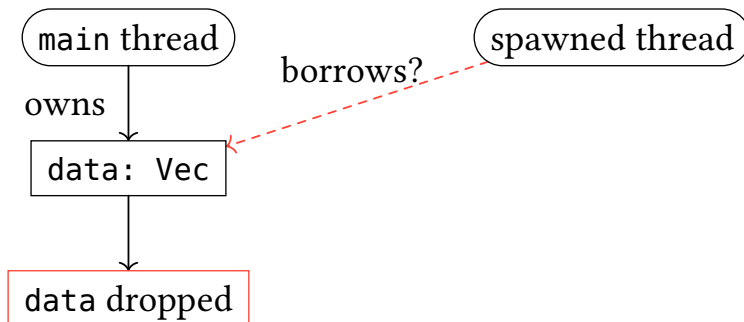
Not all references can be sent to threads.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         let sum: i32 = data.iter().sum();  
5         println!("Sum: {sum}");  
6     });  
7     data.push(4); // modify while thread may be reading  
8     handle.join().unwrap();  
9 }
```

[\(playground link\)](#)

What happens if a thread borrows data from the main thread?

Compilation error: closure may outlive the current function, but it borrows data.



1.4. Sending references to threads

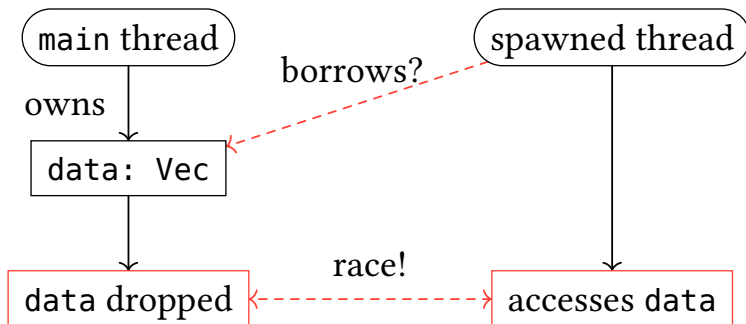
Not all references can be sent to threads.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         let sum: i32 = data.iter().sum();  
5         println!("Sum: {sum}");  
6     });  
7     data.push(4); // modify while thread may be reading  
8     handle.join().unwrap();  
9 }
```

([playground link](#))

What happens if a thread borrows data from the main thread?

Compilation error: closure may outlive the current function, but it borrows data.



1.5. Borrowing and threads (continued)

First attempt: ensure the thread finishes before main exits.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         let sum: i32 = data.iter().sum();  
5         println!("Sum: {sum}");  
6     });  
7     handle.join().unwrap();  
8     println!("Original: {:?}", data); // try to use after thread  
9 }
```

(playground link)

Does adding `join()` fix the borrow error?

1.5. Borrowing and threads (continued)

First attempt: ensure the thread finishes before main exits.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         let sum: i32 = data.iter().sum();  
5         println!("Sum: {sum}");  
6     });  
7     handle.join().unwrap();  
8     println!("Original: {:?}", data); // try to use after thread  
9 }
```

(playground link)

Does adding `join()` fix the borrow error?

No. Rust cannot verify at compile time that `join` is called before `data` goes out of scope.



1.5. Borrowing and threads (continued)

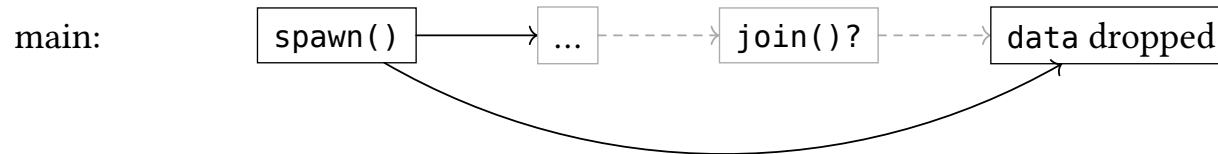
First attempt: ensure the thread finishes before main exits.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         let sum: i32 = data.iter().sum();  
5         println!("Sum: {sum}");  
6     });  
7     handle.join().unwrap();  
8     println!("Original: {:?}", data); // try to use after thread  
9 }
```

(playground link)

Does adding `join()` fix the borrow error?

No. Rust cannot verify at compile time that `join` is called before `data` goes out of scope.



1.5. Borrowing and threads (continued)

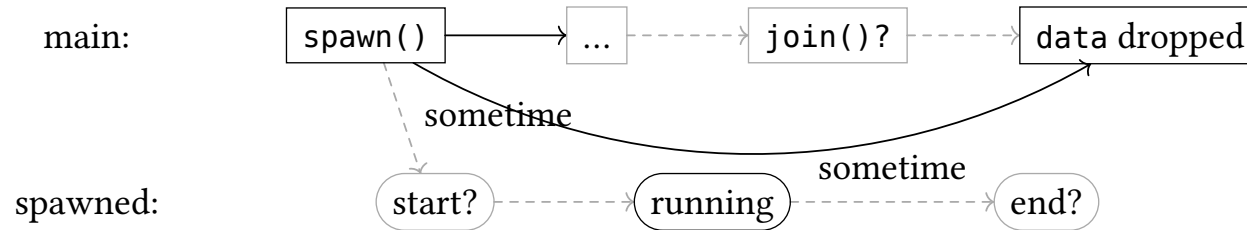
First attempt: ensure the thread finishes before main exits.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         let sum: i32 = data.iter().sum();  
5         println!("Sum: {sum}");  
6     });  
7     handle.join().unwrap();  
8     println!("Original: {:?}", data); // try to use after thread  
9 }
```

(playground link)

Does adding `join()` fix the borrow error?

No. Rust cannot verify at compile time that `join` is called before `data` goes out of scope.



1.5. Borrowing and threads (continued)

First attempt: ensure the thread finishes before main exits.

```

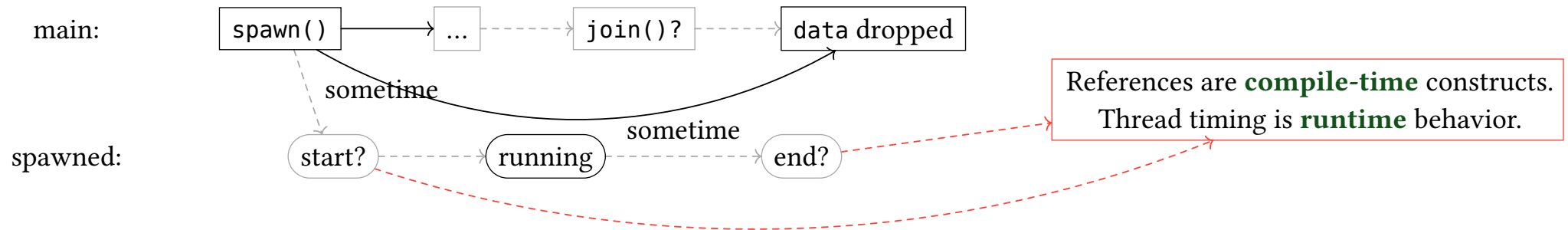
1 fn main() {
2     let data = vec![1, 2, 3];
3     let handle = thread::spawn(|| {
4         let sum: i32 = data.iter().sum();
5         println!("Sum: {sum}");
6     });
7     handle.join().unwrap();
8     println!("Original: {:?}", data); // try to use after thread
9 }

```

(playground link)

Does adding `join()` fix the borrow error?

No. Rust cannot verify at compile time that `join` is called before `data` goes out of scope.



1.6. Scoped threads

The problem: `thread::spawn` requires `'static` because the thread may outlive the caller.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data); // error: `data` does not live long enough  
5     });  
6     handle.join().unwrap();  
7 }
```

([playground link](#))

1.6. Scoped threads

The problem: `thread::spawn` requires `'static` because the thread may outlive the caller.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data); // error: `data` does not live long enough  
5     });  
6     handle.join().unwrap();  
7 }
```

[\(playground link\)](#)

Solution: `thread::scope` guarantees all spawned threads complete before the scope ends.

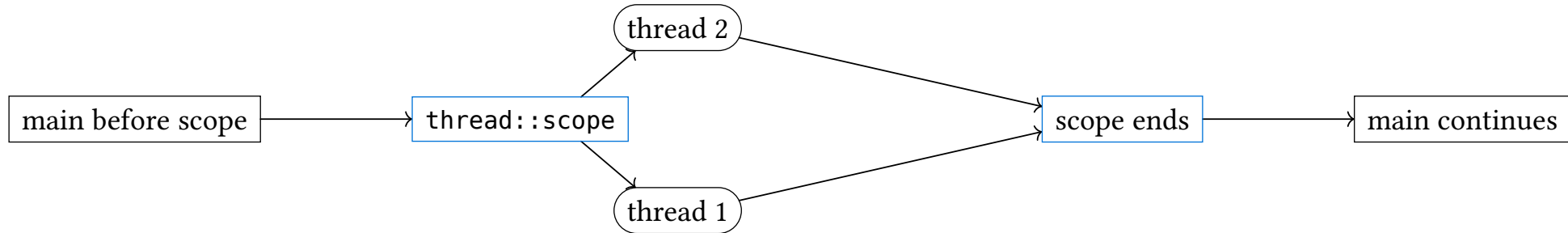
```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     thread::scope(|s| {  
4         s.spawn(|| println!("Data: {:?}", data)); // borrowing works!  
5     }); // all threads joined here automatically  
6     println!("Back in main: {:?}", data);  
7 }
```

[\(playground link\)](#)

1.7. Multiple scoped threads

```
1  fn main() {  
2      let mut data = vec![1, 2, 3];  
3  
4      thread::scope(|s| {  
5          s.spawn(|| println!("Thread 1: {:?}", data)); // shared borrow  
6          s.spawn(|| println!("Thread 2: {:?}", data)); // shared borrow  
7      });  
8  
9      data.push(4); // mutable access after scope ends  
10 }
```

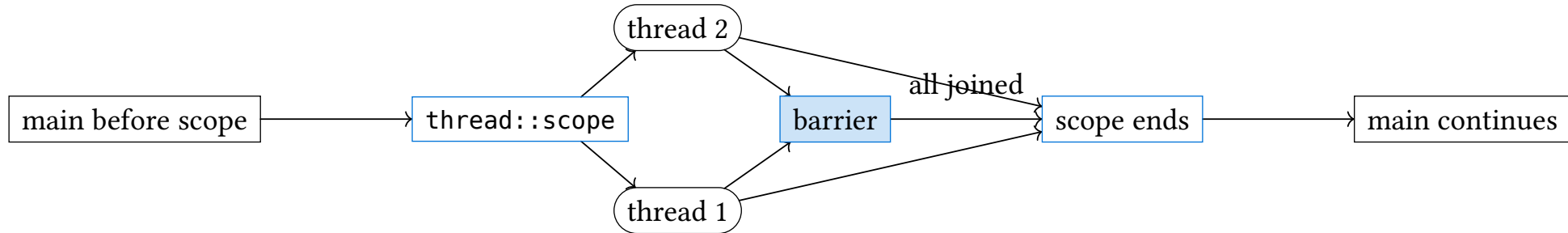
(playground link)



1.7. Multiple scoped threads

```
1  fn main() {  
2      let mut data = vec![1, 2, 3];  
3  
4      thread::scope(|s| {  
5          s.spawn(|| println!("Thread 1: {:?}", data)); // shared borrow  
6          s.spawn(|| println!("Thread 2: {:?}", data)); // shared borrow  
7      });  
8  
9      data.push(4); // mutable access after scope ends  
10 }
```

(playground link)



1.8. Moving to threads

In practice, scoped threads are not used very often.

The most common use case is to move ownership of data to the thread using the `move` keyword to transfer ownership at spawn time

```
1 fn main() {  
2     let data = String::from("hello");  
3     let handle = thread::spawn(move || {  
4         println!("Data: {data}"); // `data` moved into closure, now owned by thread  
5     });  
6     handle.join().unwrap();  
7 }
```

([playground link](#))

Move can also be used for blocks (not closures):

1.9. Review: what is 'static?

In the past sessions we have seen that `T: 'static` means:

Definition 1.9.1. *A type `T` is 'static if it contains no non-'static references.*

Things that are 'static:

- Owned data: `String`, `Vec<T>`, `Box<T>`, etc.
- 'static references: `&'static str`, `&'static T`

Things that are not 'static:

- References with shorter lifetimes: `&T`, `&'a T` where `'a` is not 'static
- Types with non-'static fields: `&T`, `&'a T`, `Vec<&T>`, etc.

1.9. Review: what is 'static?

In the past sessions we have seen that `T: 'static` means:

Definition 1.9.1. *A type `T` is 'static if it contains no non-'static references.*

Things that are 'static:

- Owned data: `String`, `Vec<T>`, `Box<T>`, etc.
- 'static references: `&'static str`, `&'static T`

Things that are not 'static:

- References with shorter lifetimes: `&T`, `&'a T` where `'a` is not 'static
- Types with non-'static fields: `&T`, `&'a T`, `Vec<&T>`, etc.

Strings are owned and without references, so they are 'static and can be moved

1.9. Review: what is 'static?

In the past sessions we have seen that T: 'static means:

Definition 1.9.1. *A type T is 'static if it contains no non-'static references.*

Things that are 'static:

- Owned data: String, Vec<T>, Box<T>, etc.
- 'static references: &'static str, &'static T

Things that are not 'static:

- References with shorter lifetimes: &T, &'a T where 'a is not 'static
- Types with non-'static fields: &T, &'a T, Vec<&T>, etc.

Strings are owned and without references, so they are 'static and can be moved

1.9.1. Spawn function

The standard library spawn function looks like this (simplified):

```
1 fn spawn<F, T>(f: F) -> JoinHandle<T>
2 where
3     F: FnOnce() -> T + 'static,
4     T: 'static
```

([playground link](#))

(Similar for asynchronous spawns like tokio::spawn, see next session.)

1. Threads	2
2. Channels	13
2.1. What is a channel?	14
2.2. N-to-1 Channels	15
2.3. Bounded channels	16
2.4. Exercises	17
3. Send and Sync	18
4. Shared state	29
5. Mutexes	33

2.1. What is a channel?

A channel is a communication primitive for passing messages between threads.

Under the hood: a thread-safe buffer (queue) in shared memory with:

- **Sender** (tx): pushes messages into the buffer
- **Receiver** (rx): pulls messages from the buffer



2.1. What is a channel?

A channel is a communication primitive for passing messages between threads.

Under the hood: a thread-safe buffer (queue) in shared memory with:

- **Sender** (tx): pushes messages into the buffer
- **Receiver** (rx): pulls messages from the buffer

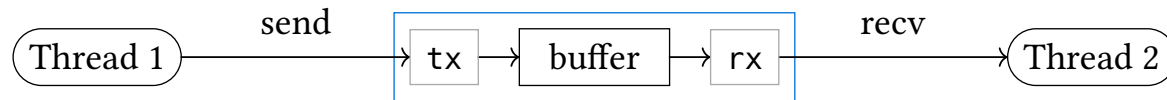


2.1. What is a channel?

A channel is a communication primitive for passing messages between threads.

Under the hood: a thread-safe buffer (queue) in shared memory with:

- **Sender** (tx): pushes messages into the buffer
- **Receiver** (rx): pulls messages from the buffer



Key properties:

- **Decouples** sender and receiver (no shared mutable state)
- **Asynchronous**: sender doesn't wait for receiver (unless bounded)
- **Ownership transfer**: messages are moved, not shared

2.2. N-to-1 Channels

```
1  fn main() {
2      let (tx, rx) = std::sync::mpsc::channel();
3      std::thread::spawn(move || {
4          let thread_id = std::thread::current().id();
5          for i in 0..10 {
6              tx.send(format!("Message {i}")).unwrap();
7              println!("{thread_id:?}: sent Message {i}");
8          }
9          println!("{thread_id:?}: done");
10     });
11     std::thread::sleep(std::time::Duration::from_millis(100));
12
13     for msg in rx.iter() {
14         println!("Main: got {msg}");
15     }
16 }
```

(playground link)

In an unbounded channel:

- `send()` never blocks (a kind of “asynchronous” behavior)
- returns a `Result::Err` when all receivers have been dropped

Definition 2.2.1. A channel is “closed” when all receivers have been dropped.

2.3. Bounded channels

If you don't want the channel buffer to grow indefinitely and cause memory overflow, you can use a bounded channel:

2.3. Bounded channels

If you don't want the channel buffer to grow indefinitely and cause memory overflow, you can use a bounded channel:

```
1  fn main() {
2      let (tx, rx) = mpsc::sync_channel(3);
3
4      thread::spawn(move || {
5          let thread_id = thread::current().id();
6          for i in 0..10 {
7              tx.send(format!("Message {i}")).unwrap();
8              println!("{thread_id:?}: sent Message {i}");
9          }
10         println!("{thread_id:?}: done");
11     });
12     thread::sleep(Duration::from_millis(100));
13
14     for msg in rx.iter() {
15         println!("Main: got {msg}");
16     }
17 }
```

(playground link)

This channel provides an additional method `try_send()` that returns immediately with an error if the buffer is full.

2.4. Exercises

- `examples/channel.rs`

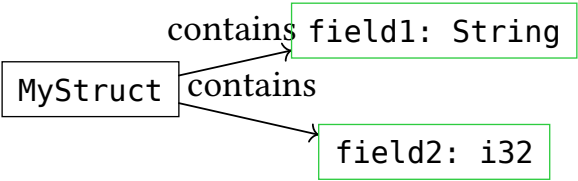
1. Threads	2
2. Channels	13
3. Send and Sync	18
3.1. Marker Traits	19
3.2. Send and Sync	20
3.3. Send + Sync	21
3.4. Atomics	22
3.5. Atomics (continued)	23
3.6. Atomics: hardware support	24
3.7. Send + !Sync	25
3.8. !Send + Sync	26
3.9. !Send + !Sync	27
3.10. Summary	28
4. Shared state	29
5. Mutexes	33

What are auto traits?

3.1. Marker Traits

What are auto traits?

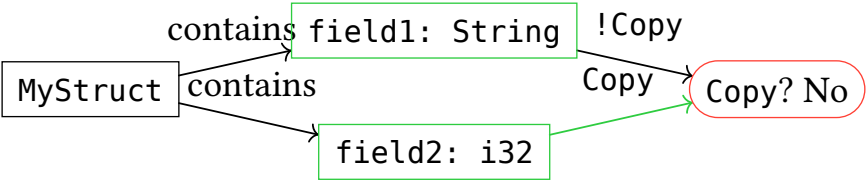
The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



3.1. Marker Traits

What are auto traits?

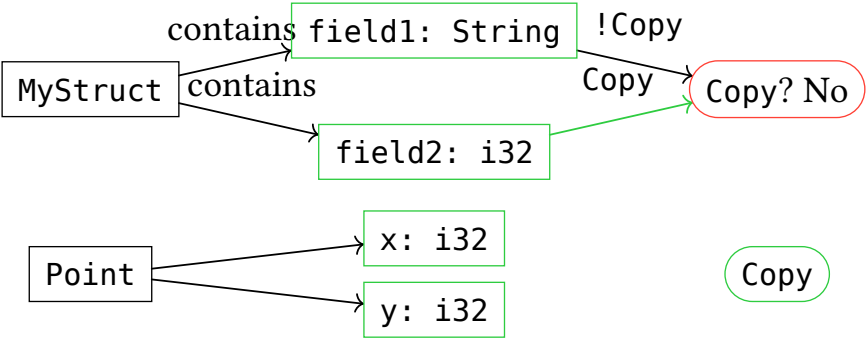
The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



3.1. Marker Traits

What are auto traits?

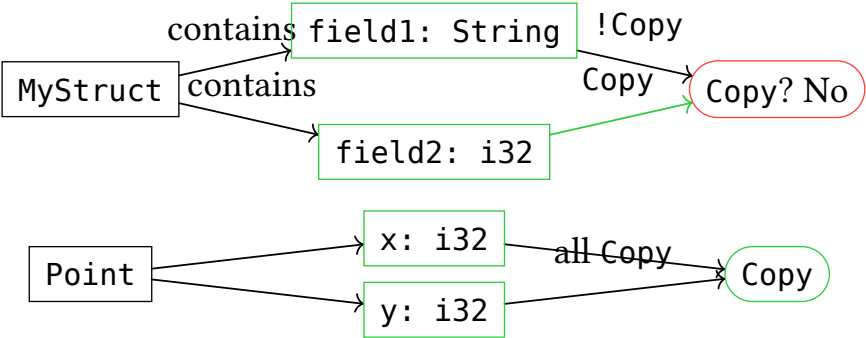
The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



3.1. Marker Traits

What are auto traits?

The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.

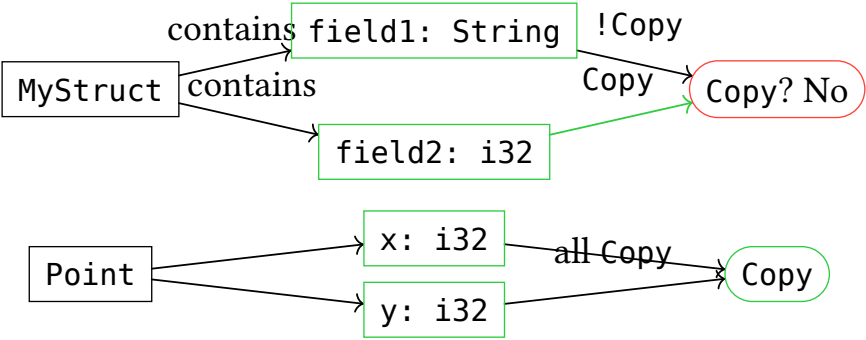


What are unsafe traits?

3.1. Marker Traits

What are auto traits?

The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



What are unsafe traits?

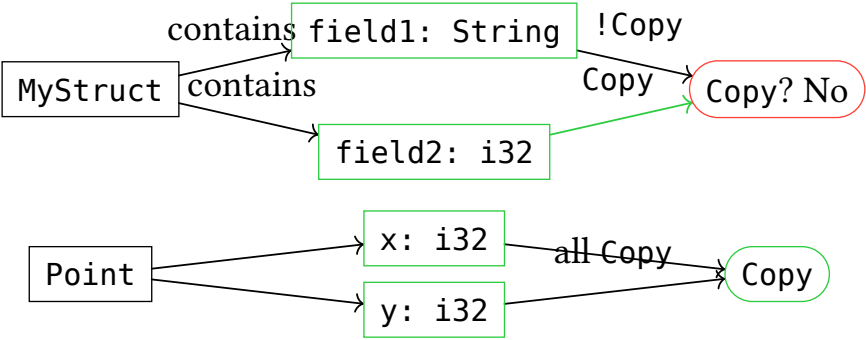
Traits that have safety invariants that the compiler cannot verify automatically. Implementing them incorrectly can lead to undefined behavior.

You can implement unsafe traits manually when you know it is valid.

3.1. Marker Traits

What are auto traits?

The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



What are unsafe traits?

Traits that have safety invariants that the compiler cannot verify automatically. Implementing them incorrectly can lead to undefined behavior.

You can implement unsafe traits manually when you know it is valid.

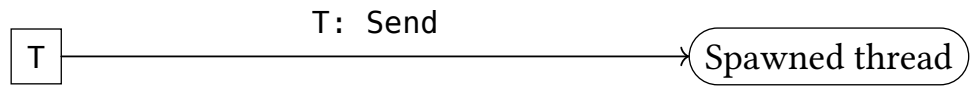
Warning

Implementing unsafe traits requires unsafe blocks. Using them as constraints does not.

3.2. Send and Sync

Definition 3.2.1.

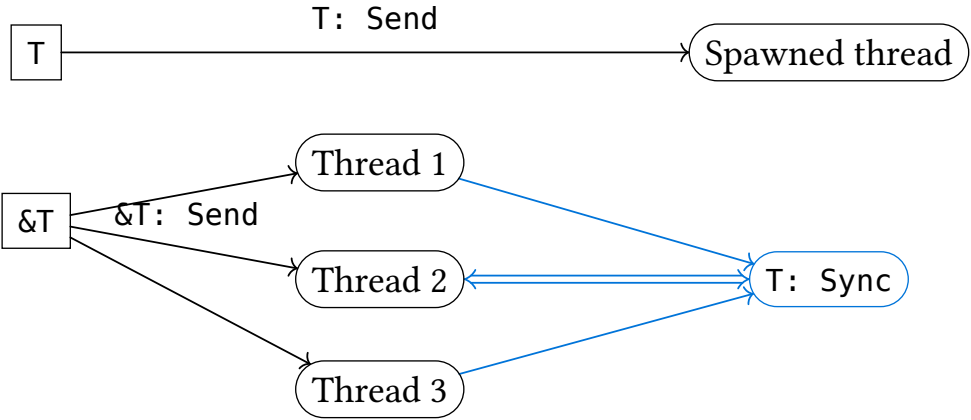
- *Send*: safe to **move** T to another thread
- *Sync*: safe to **share** $\&T$ across threads (i.e., $\&T$: *Send*)



3.2. Send and Sync

Definition 3.2.1.

- Send: safe to **move** *T* to another thread
- Sync: safe to **share** *&T* across threads (i.e., *&T*: Send)



Info

Send and Sync are unsafe auto traits.

3.3. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** `i32`, `bool`, `String`, `Vec<T>`

3.3. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership:** `Arc<T>` (atomic ref counting)

3.3. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership:** `Arc<T>` (atomic ref counting)
- **Hardware-supported atomics:** `AtomicBool`, `AtomicU8` (compare-and-swap, fetch-and-add)

When are the generic types such as `Vec<T>` Send + Sync

3.3. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership:** `Arc<T>` (atomic ref counting)
- **Hardware-supported atomics:** `AtomicBool`, `AtomicU8` (compare-and-swap, fetch-and-add)

When are the generic types such as `Vec<T>` Send + Sync

When the type parameters `T` are Send + Sync.

3.4. Atomics

Atomics are low-level types for lock-free concurrent programming.

3.4. Atomics

Atomics are low-level types for lock-free concurrent programming.

Two problems with normal memory operations:

- Compilers and CPUs may **reorder** instructions for optimization
- Read-modify-write (e.g., `x += 1`) is **not atomic** (multiple CPU instructions)

3.4. Atomics

Atoms are low-level types for lock-free concurrent programming.

Two problems with normal memory operations:

- Compilers and CPUs may **reorder** instructions for optimization
- Read-modify-write (e.g., `x += 1`) is **not atomic** (multiple CPU instructions)

```
1 static mut DATA: u32 = 0;
2 static mut READY: bool = false;
3
4 fn thread1() { unsafe { DATA = 42; READY = true; } }
5 fn thread2() { unsafe { while !READY {} println!("{}", DATA); } } // might print 0! (playground link)
```

CPU might execute `READY = true` before `DATA = 42`.

3.5. Atomics (continued)

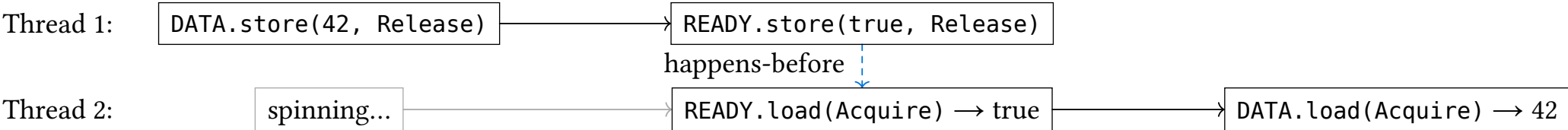
Solution: atomic operations with memory ordering guarantees.

```

1  use std::sync::atomic::{AtomicBool, AtomicU32, Ordering};
2
3  static DATA: AtomicU32 = AtomicU32::new(0);
4  static READY: AtomicBool = AtomicBool::new(false);
5
6  fn thread1() {
7      DATA.store(42, Ordering::Release);
8      READY.store(true, Ordering::Release);
9  }
10 fn thread2() {
11     while !READY.load(Ordering::Acquire) {}
12     println!("{}", DATA.load(Ordering::Acquire)); // guaranteed 42
13 }

```

(playground link)



The happens-before edge is on READY (same atomic, Release-store → Acquire-load).

This transitively makes DATA.store visible to DATA.load.

3.6. Atomics: hardware support

Not all CPUs support all atomic operations natively.

3.6. Atomics: hardware support

Not all CPUs support all atomic operations natively.

- **Always supported:** AtomicBool, AtomicU8, AtomicU16, AtomicU32, AtomicUsize
- **64-bit only:** AtomicU64 (not available on 32-bit targets)
- **Rare:** AtomicU128 (only some x86-64 CPUs)

3.6. Atomics: hardware support

Not all CPUs support all atomic operations natively.

- **Always supported:** AtomicBool, AtomicU8, AtomicU16, AtomicU32, AtomicUsize
- **64-bit only:** AtomicU64 (not available on 32-bit targets)
- **Rare:** AtomicU128 (only some x86-64 CPUs)

Check at compile time:

```
1 #[cfg(target_has_atomic = "64")]  
2 use std::sync::atomic::AtomicU64;
```

([playground link](#))

3.6. Atomics: hardware support

Not all CPUs support all atomic operations natively.

- **Always supported:** AtomicBool, AtomicU8, AtomicU16, AtomicU32, AtomicUsize
- **64-bit only:** AtomicU64 (not available on 32-bit targets)
- **Rare:** AtomicU128 (only some x86-64 CPUs)

Check at compile time:

```
1 #[cfg(target_has_atomic = "64")]  
2 use std::sync::atomic::AtomicU64;
```

[\(playground link\)](#)

If hardware doesn't support an atomic size, Rust either:

- Won't compile (type not available)
- Falls back to a lock-based implementation (slower)

Info

On modern x86-64 and ARM64, all common atomics are hardware-supported.

O'REILLY®

Rust Atomics and Locks

Low-Level Concurrency in Practice



Mara Bos

3.7. Send + !Sync

These types can be moved to other threads, but cannot be shared via &T:

3.7. Send + !Sync

These types can be moved to other threads, but cannot be shared via &T:

- **Non-thread-safe interior mutability:** `Cell<T>`, `RefCell<T>`
 - Allow mutation through &T without synchronization
 - Safe to own exclusively on one thread, unsafe to share references

3.8. !Send + Sync

These types are safe to access (via shared references) from multiple threads, but cannot be moved to another thread:

- **Lock guards:** `MutexGuard`, `RwLockReadGuard`, `RwLockWriteGuard`
 - Must be unlocked on the thread that acquired the lock (POSIX requirement)

Proposition 3.8.1. *`MutexGuard<T: Sync>: !Send + Sync`*

3.9. !Send + !Sync

These types are not thread-safe and cannot be moved to other threads:

3.9. !Send + !Sync

These types are not thread-safe and cannot be moved to other threads:

- dereferences and has non-atomic count: `Rc<T>`
- considered unsafe in general `*const T`, `*mut T`

	Sync	!Sync
Send	i32, bool, String, Vec<T> Arc<T>, Mutex<T>, Atomic* Safe to move and share	Cell<T>, RefCell<T> Interior mutability: safe to move, not share
!Send	MutexGuard, RwLock*Guard Thread-bound: safe to share, not move	Rc<T>, *const T, *mut T Not thread-safe at all

1. Threads	2
2. Channels	13
3. Send and Sync	18
4. Shared state	29
4.1. Sharing data	30
4.2. Concept	31
5. Mutexes	33

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;  
2 fn increment() {  
3     unsafe {  
4         COUNTER += 1;  
5     }  
6 }
```

([playground link](#))

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;  
2 fn increment() {  
3     unsafe {  
4         COUNTER += 1;  
5     }  
6 }
```

(playground link)

`static mut COUNTER`

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;  
2 fn increment() {  
3     unsafe {  
4         COUNTER += 1;  
5     }  
6 }
```

(playground link)

Stack (Thread 1)

unsafe

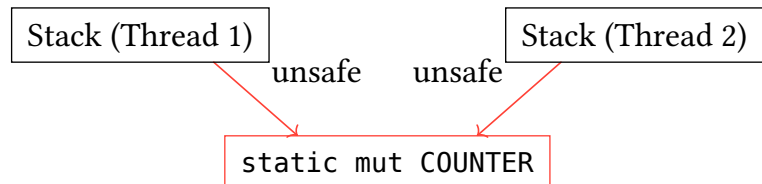
static mut COUNTER

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;  
2 fn increment() {  
3     unsafe {  
4         COUNTER += 1;  
5     }  
6 }
```

(playground link)

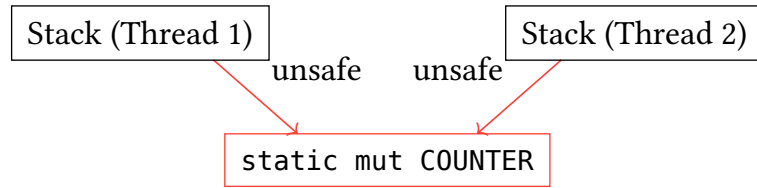


4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;  
2 fn increment() {  
3     unsafe {  
4         COUNTER += 1;  
5     }  
6 }
```

(playground link)

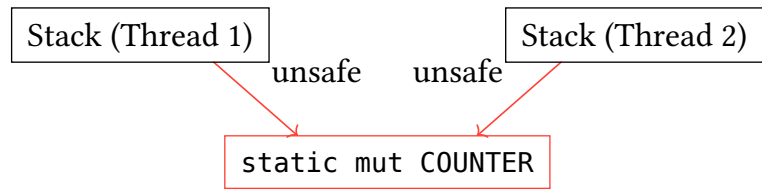


4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;
2 fn increment() {
3     unsafe {
4         COUNTER += 1;
5     }
6 }
```

(playground link)



Shared memory,
no synchronization!

Disadvantages:

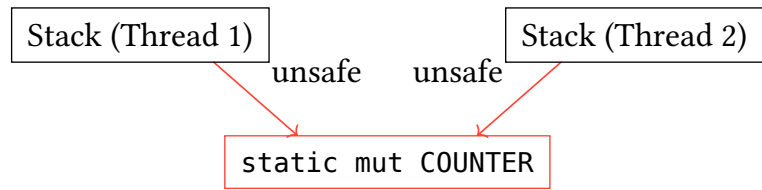
- Unsafe: requires `unsafe` blocks to access
- Needs to be initialized for the entire program lifetime

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;
2 fn increment() {
3     unsafe {
4         COUNTER += 1;
5     }
6 }
```

(playground link)



Shared memory,
no synchronization!

Disadvantages:

- Unsafe: requires `unsafe` blocks to access
- Needs to be initialized for the entire program lifetime

Solution:

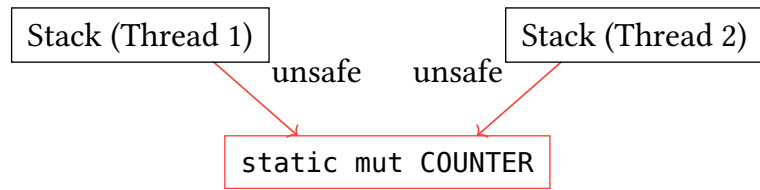
1. Use reference counted variable (next slides)

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;
2 fn increment() {
3     unsafe {
4         COUNTER += 1;
5     }
6 }
```

(playground link)



Shared memory,
no synchronization!

Disadvantages:

- Unsafe: requires `unsafe` blocks to access
- Needs to be initialized for the entire program lifetime

Solution:

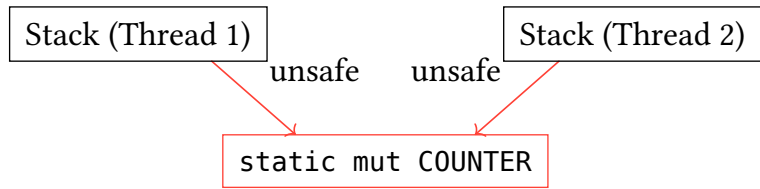
1. Use reference counted variable (next slides)
2. Use a lock to ensure exclusive access (later)

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;
2 fn increment() {
3     unsafe {
4         COUNTER += 1;
5     }
6 }
```

([playground link](#))



Shared memory,
no synchronization!

Disadvantages:

- Unsafe: requires `unsafe` blocks to access
- Needs to be initialized for the entire program lifetime

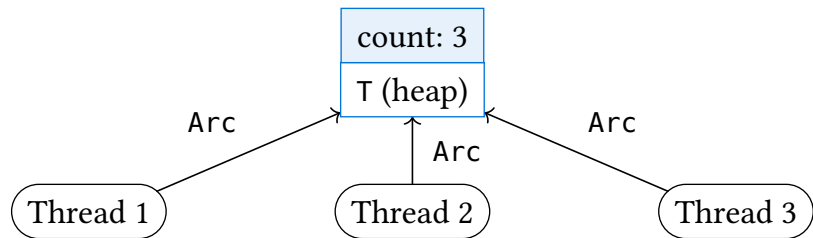
Solution:

1. Use reference counted variable (next slides)
2. Use a lock to ensure exclusive access (later)

(Notice that in this simple case we could just use an `AtomicU32` instead. This is just an example.)

4.2. Concept

Definition 4.2.1. $\text{Arc}\langle T \rangle = \text{Atomic Reference Counted}$ pointer for shared ownership across threads.



Key properties:

- `Arc::clone(&v)` increments ref count (atomic operation)
- When count reaches 0, data is dropped
- `Arc<T>`: Send + Sync when `T`: Send + Sync

4.2. Concept

4.2.1. Arc example

```
1  use std::sync::Arc;
2  use std::thread;
3
4  fn main() {
5      let data = Arc::new(vec![1, 2, 3]);
6
7      let handles: Vec<_> = (0..3).map(|i| {
8          let data = Arc::clone(&data);
9          thread::spawn(move || {
10             println!("Thread {i}: {:?}", data);
11         })
12     }).collect();
13
14     for h in handles { h.join().unwrap(); }
15 }
```

(playground link)

4.2. Concept

4.2.1. Arc example

```
1  use std::sync::Arc;
2  use std::thread;
3
4  fn main() {
5      let data = Arc::new(vec![1, 2, 3]);
6
7      let handles: Vec<_> = (0..3).map(|i| {
8          let data = Arc::clone(&data);
9          thread::spawn(move || {
10             println!("Thread {i}: {:?}", data);
11         })
12     }).collect();
13
14     for h in handles { h.join().unwrap(); }
15 }
```

(playground link)

Who drops the data?

4.2. Concept

4.2.1. Arc example

```
1 use std::sync::Arc;
2 use std::thread;
3
4 fn main() {
5     let data = Arc::new(vec![1, 2, 3]);
6
7     let handles: Vec<_> = (0..3).map(|i| {
8         let data = Arc::clone(&data);
9         thread::spawn(move || {
10             println!("Thread {i}: {:?}", data);
11         })
12     }).collect();
13
14     for h in handles { h.join().unwrap(); }
15 }
```

(playground link)

Who drops the data?

The last thread to finish (last Arc clone to go out of scope).

Can we mutate T through Arc<T>?

4.2. Concept

4.2.1. Arc example

```
1 use std::sync::Arc;
2 use std::thread;
3
4 fn main() {
5     let data = Arc::new(vec![1, 2, 3]);
6
7     let handles: Vec<_> = (0..3).map(|i| {
8         let data = Arc::clone(&data);
9         thread::spawn(move || {
10             println!("Thread {i}: {:?}", data);
11         })
12     }).collect();
13
14     for h in handles { h.join().unwrap(); }
15 }
```

([playground link](#))

Who drops the data?

The last thread to finish (last Arc clone to go out of scope).

Can we mutate T through Arc<T>?

No, Arc only provides shared access. Use Arc<Mutex<T>> for mutation.

1. Threads	2
2. Channels	13
3. Send and Sync	18
4. Shared state	29
5. Mutexes	33
5.1. Mutex	34
5.2. Interior mutability	37
5.3. Deadlocks	38
5.4. Origins of deadlocks	39
5.5. Dining Philosophers deadlock	40
5.6. Best practices for Mutexes	42

5.1. Mutex

Prevents race conditions on complex shared data using synchronisation.

How are you used to do data synchronisation in your favourite language?

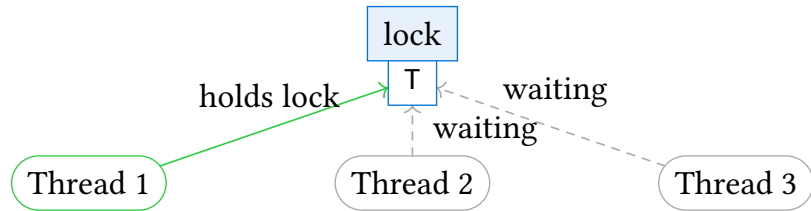
5.1. Mutex

Prevents race conditions on complex shared data using synchronisation.

How are you used to do data synchronisation in your favourite language?

...

Definition 5.1.1. $\text{Mutex}<T>$ = **Mutual Exclusion** lock for exclusive access to T across threads.



Key operations:

- `lock()` $\rightarrow$ blocks until lock acquired, returns `MutexGuard<T>`
- `MutexGuard` auto-releases lock when dropped (RAII)
- `Mutex<T>`: Sync when `T`: Send

5.1. Mutex

5.1.1. Mutex example

```
1  use std::sync::Mutex;
2
3  fn main() {
4      let counter = Mutex::new(0);
5
6      {
7          let mut guard = counter.lock().unwrap();
8          *guard += 1;
9      } // lock released here
10
11     println!("Counter: {:?}", counter.lock().unwrap());
12 }
```

(playground link)

5.1. Mutex

5.1.1. Mutex example

```
1  use std::sync::Mutex;
2
3  fn main() {
4      let counter = Mutex::new(0);
5
6      {
7          let mut guard = counter.lock().unwrap();
8          *guard += 1;
9      } // lock released here
10
11     println!("Counter: {:?}", counter.lock().unwrap());
12 }
```

(playground link)

What is a PoisonError?

5.1. Mutex

5.1.1. Mutex example

```
1  use std::sync::Mutex;
2
3  fn main() {
4      let counter = Mutex::new(0);
5
6      {
7          let mut guard = counter.lock().unwrap();
8          *guard += 1;
9      } // lock released here
10
11     println!("Counter: {:?}", counter.lock().unwrap());
12 }
```

(playground link)

What is a PoisonError?

Another thread panicked while holding the lock. The data may be in an inconsistent state.

Why does `Mutex<T>` require `T: Send` but not `T: Sync`?

5.1. Mutex

5.1.1. Mutex example

```
1  use std::sync::Mutex;
2
3  fn main() {
4      let counter = Mutex::new(0);
5
6      {
7          let mut guard = counter.lock().unwrap();
8          *guard += 1;
9      } // lock released here
10
11     println!("Counter: {:?}", counter.lock().unwrap());
12 }
```

(playground link)

What is a PoisonError?

Another thread panicked while holding the lock. The data may be in an inconsistent state.

Why does Mutex<T> require T: Send but not T: Sync?

The mutex guarantees only one thread accesses T at a time, so T doesn't need to support concurrent access.

5.1. Mutex

5.1.2. Arc + Mutex: shared mutable state

```
1  use std::sync::{Arc, Mutex};
2  use std::thread;
3
4  fn main() {
5      let counter = Arc::new(Mutex::new(0));
6
7      let handles: Vec<_> = (0..4).map(|_| {
8          let counter = Arc::clone(&counter);
9          thread::spawn(move || {
10              let mut guard = counter.lock().unwrap();
11              *guard += 1;
12          })
13      }).collect();
14
15      for h in handles { h.join().unwrap(); }
16      println!("Counter: {}", counter.lock().unwrap()); // 4
17 }
```

[\(playground link\)](#)

- Arc provides shared ownership across threads
- Mutex provides exclusive mutable access

5.2. Interior mutability

Definition 5.2.1 (Interior mutability). *A type that allows mutation through a shared reference ($\&T \rightarrow \&\text{mut inner}$).*

5.2. Interior mutability

Definition 5.2.1 (Interior mutability). *A type that allows mutation through a shared reference ($\&T \rightarrow \&\text{mut inner}$).*

Types with interior mutability:

Type	Thread-safe?	Checking	Use case
<code>Cell<T></code>	No	Compile-time	Simple values, Copy types
<code>RefCell<T></code>	No	Runtime (panics)	Complex borrows, single-threaded
<code>Mutex<T></code>	Yes	Runtime (blocks)	Exclusive access across threads
<code>RwLock<T></code>	Yes	Runtime (blocks)	Many readers, few writers
<code>Atomic*</code>	Yes	Hardware	Counters, flags, lock-free

5.2. Interior mutability

Definition 5.2.1 (Interior mutability). *A type that allows mutation through a shared reference ($\&T \rightarrow \&mut\ inner$).*

Types with interior mutability:

Type	Thread-safe?	Checking	Use case
<code>Cell<T></code>	No	Compile-time	Simple values, Copy types
<code>RefCell<T></code>	No	Runtime (panics)	Complex borrows, single-threaded
<code>Mutex<T></code>	Yes	Runtime (blocks)	Exclusive access across threads
<code>RwLock<T></code>	Yes	Runtime (blocks)	Many readers, few writers
<code>Atomic*</code>	Yes	Hardware	Counters, flags, lock-free

5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations

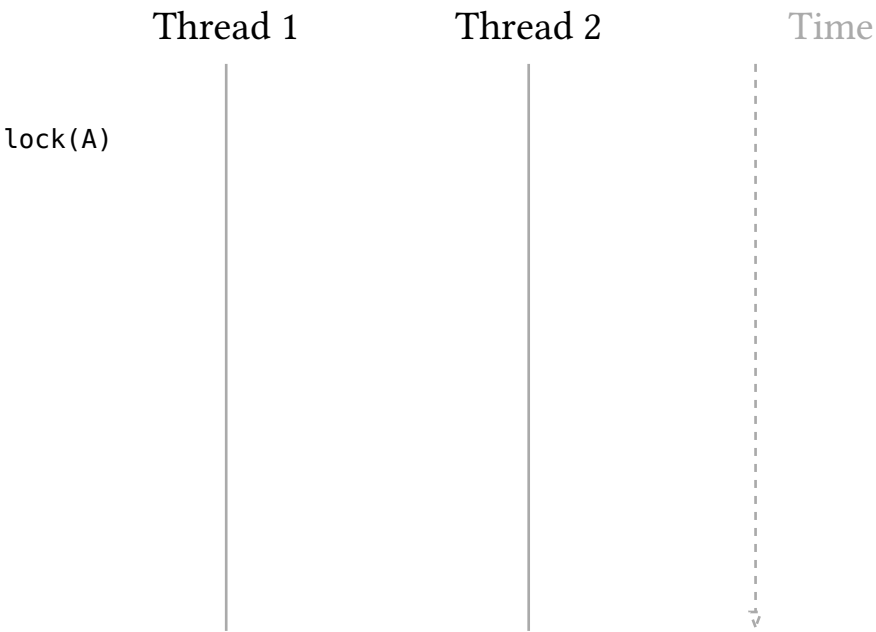
Thread 1

Thread 2

Time

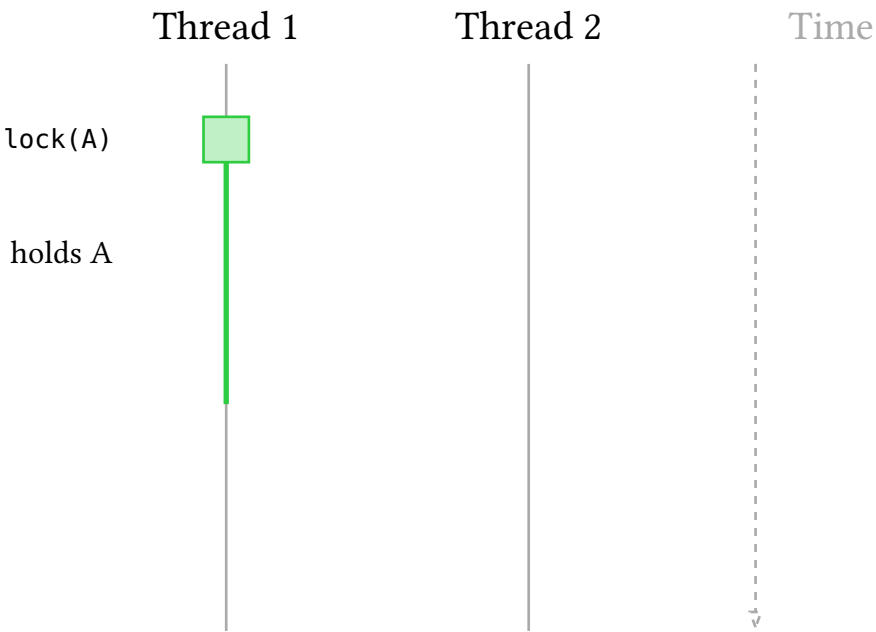
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



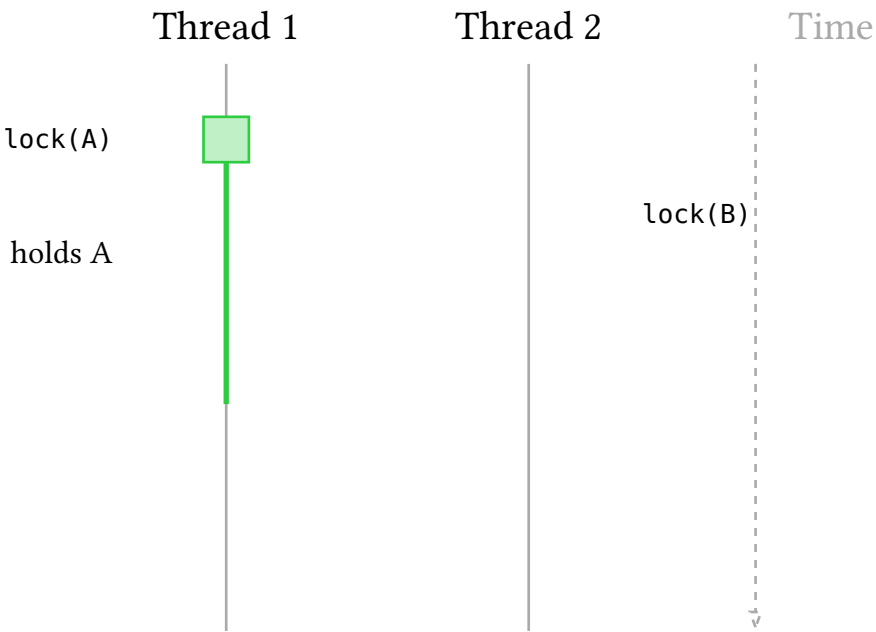
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



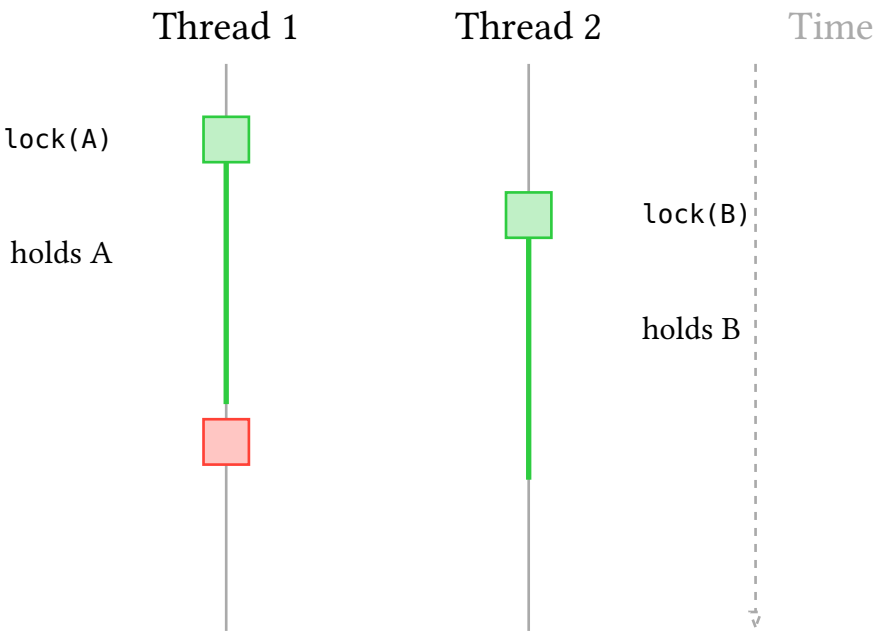
5.3. Deadlocks

The Arc<Mutex<T>> pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



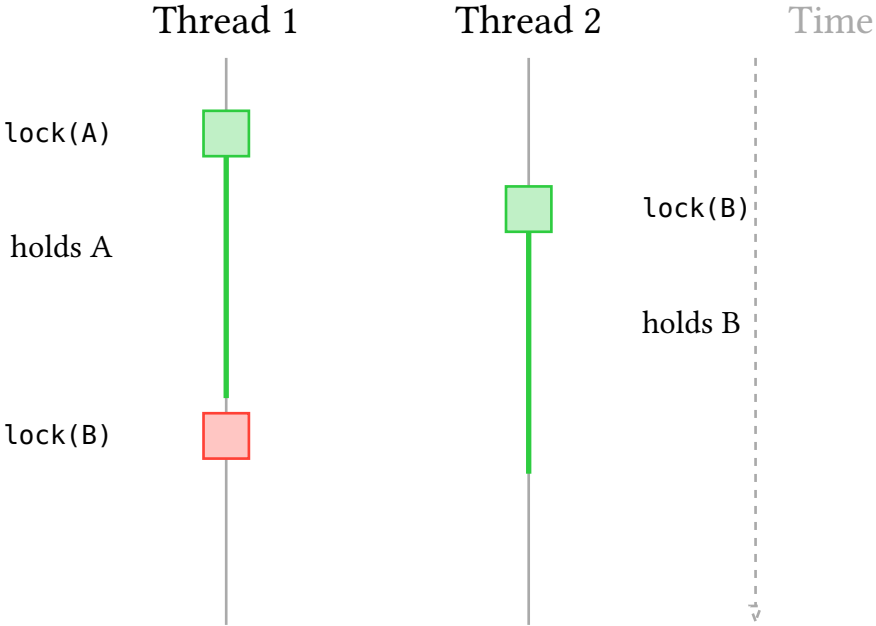
5.3. Deadlocks

The Arc<Mutex<T>> pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



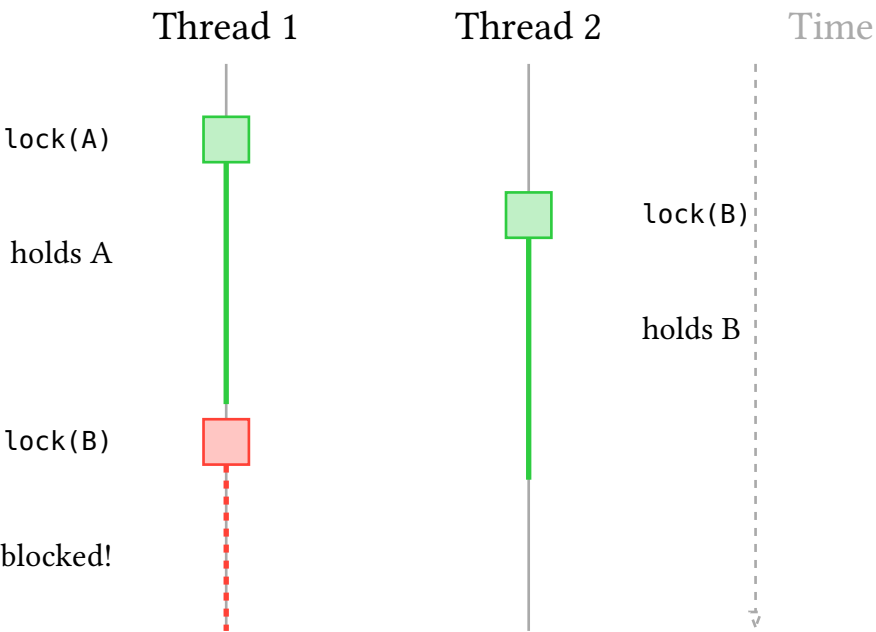
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



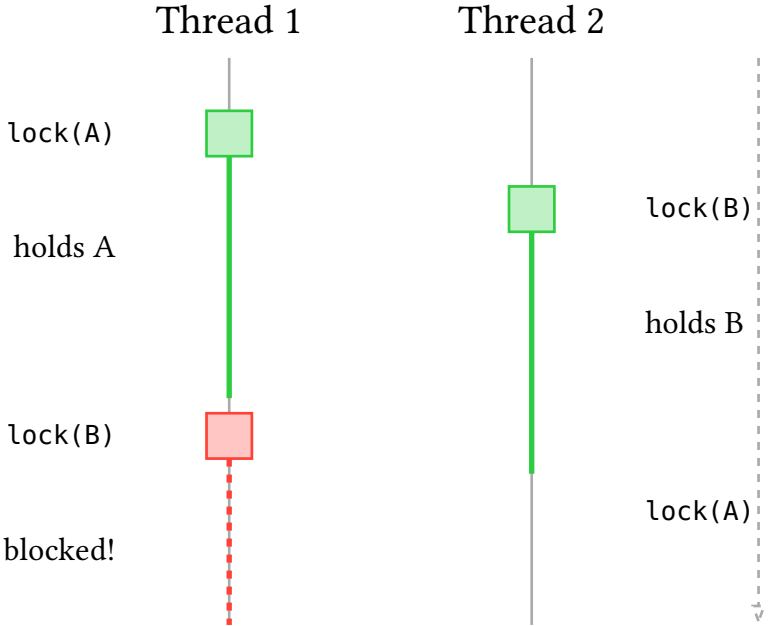
5.3. Deadlocks

The Arc<Mutex<T>> pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



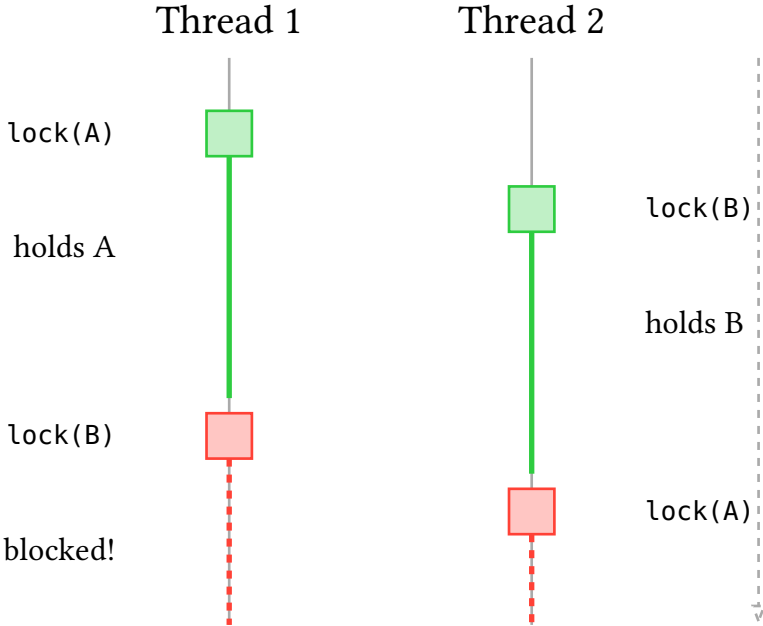
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



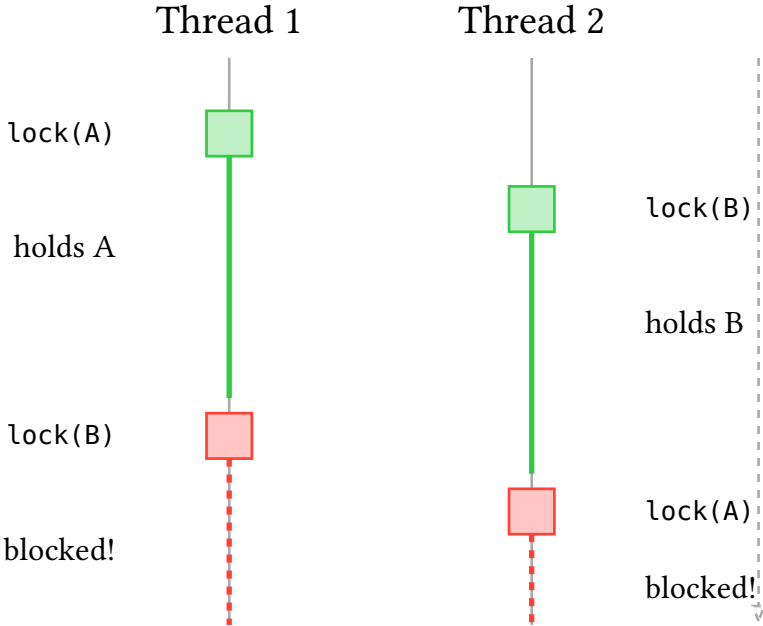
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



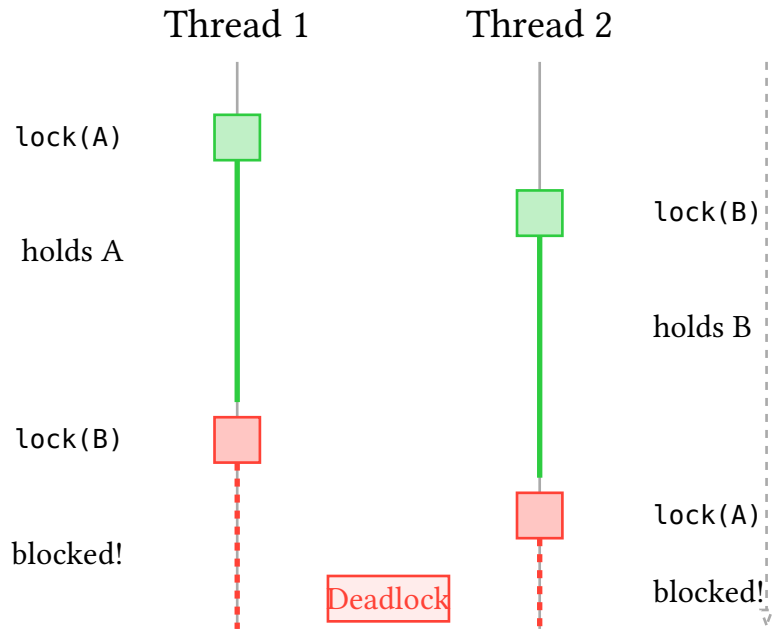
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



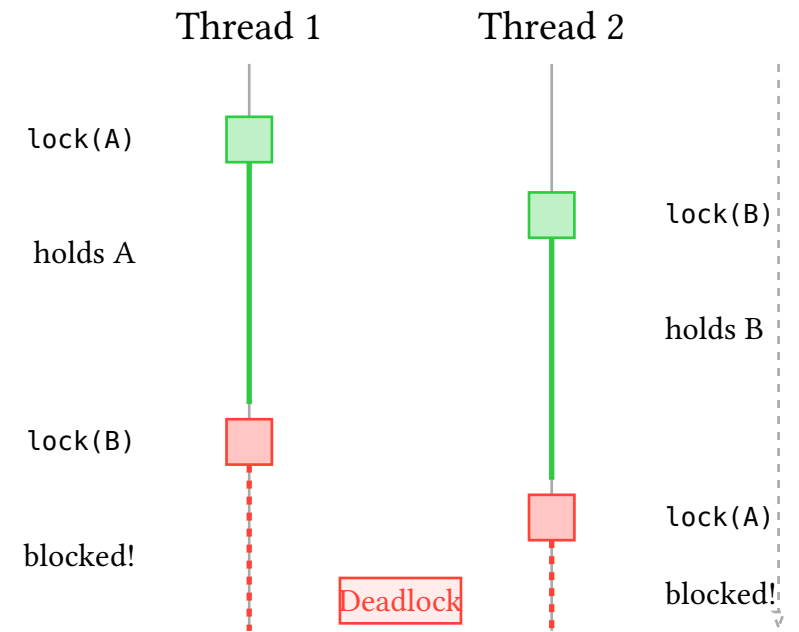
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



5.4. Origins of deadlocks

A deadlock occurs when four conditions are met simultaneously (Coffman conditions):

Condition	Description
Mutual exclusion	Resources cannot be shared (only one thread at a time)
Hold and wait	Threads hold resources while waiting for others
No preemption	Resources cannot be forcibly taken from threads
Circular wait	Threads form a cycle waiting for each other's resources

5.4. Origins of deadlocks

A deadlock occurs when four conditions are met simultaneously (Coffman conditions):

Condition	Description
Mutual exclusion	Resources cannot be shared (only one thread at a time)
Hold and wait	Threads hold resources while waiting for others
No preemption	Resources cannot be forcibly taken from threads
Circular wait	Threads form a cycle waiting for each other's resources

To prevent deadlock, break at least one condition:

- **Lock ordering:** Always acquire locks in the same order (breaks circular wait)
- **Timeout:** Release locks if waiting too long (breaks hold and wait)
- **Try-lock:** Use `try_lock()` instead of `lock()` (breaks hold and wait)

5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

Chopstick 2

Chopstick 0

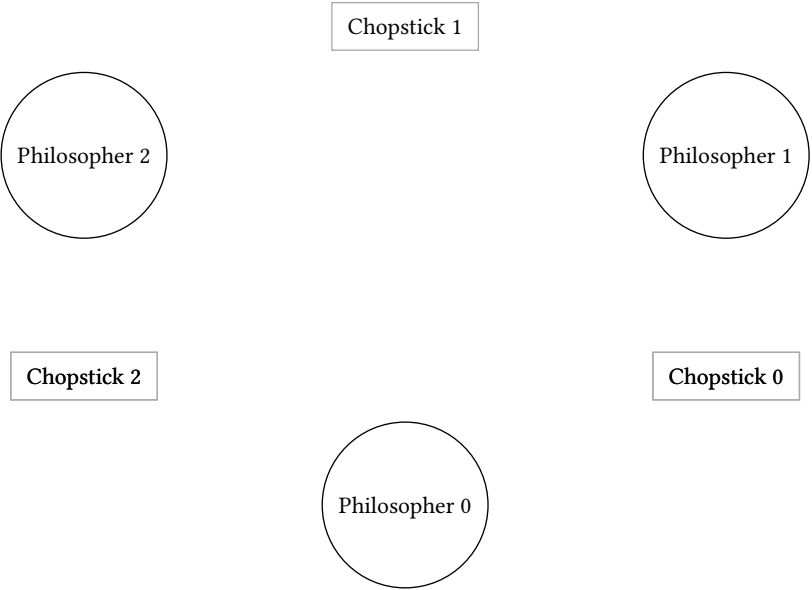
Philosopher 0

5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

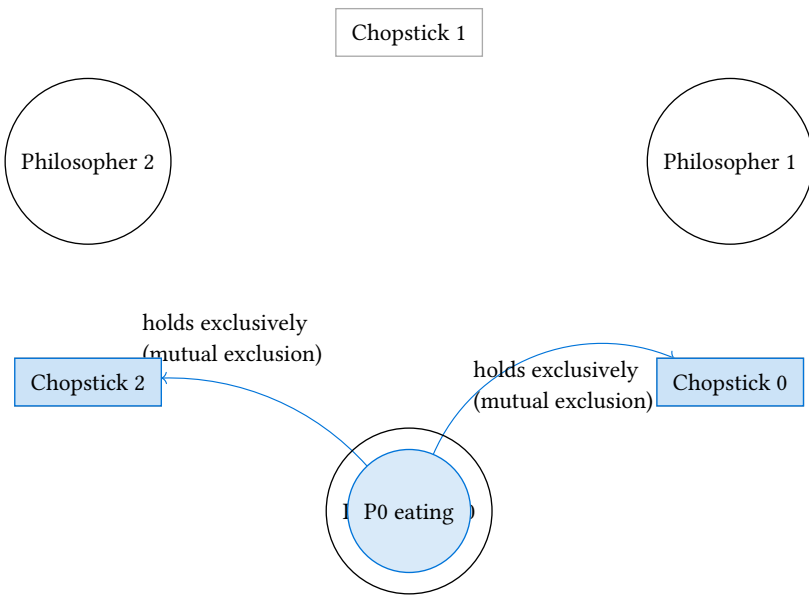


5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

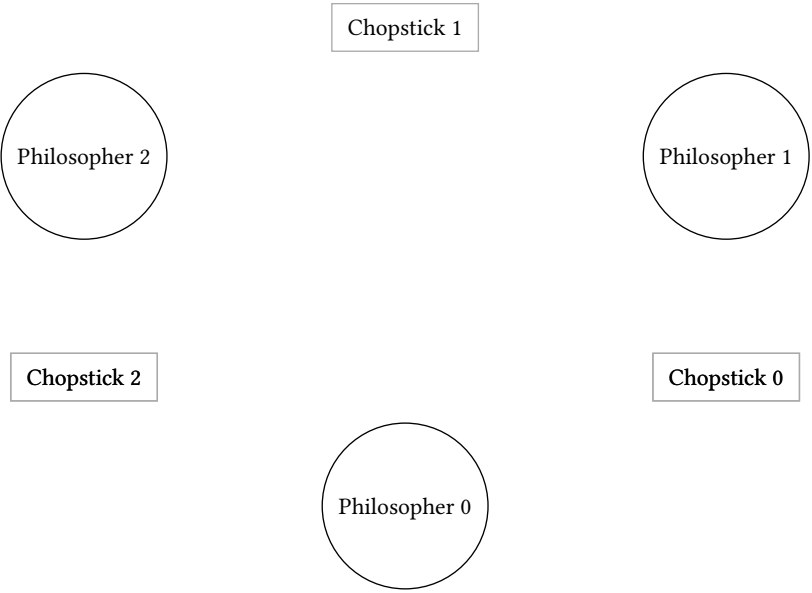


5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

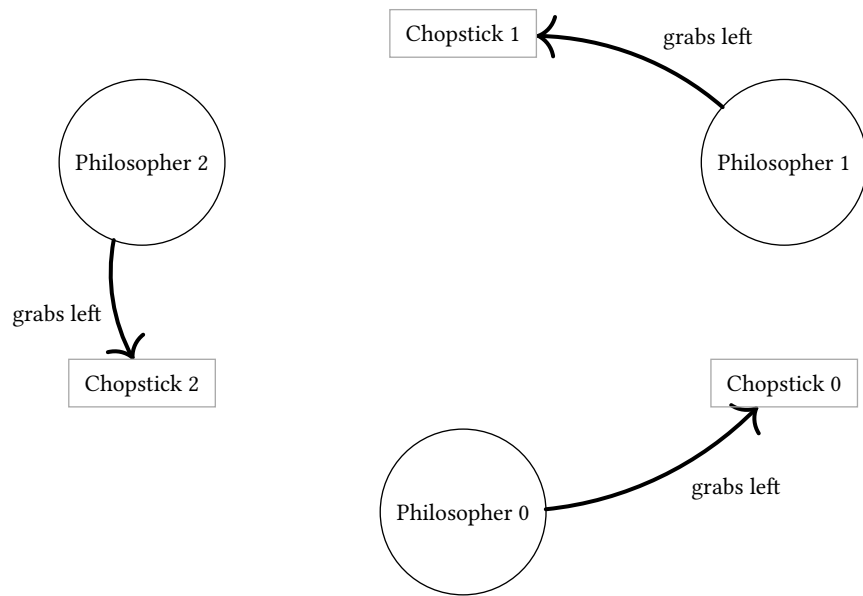


5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

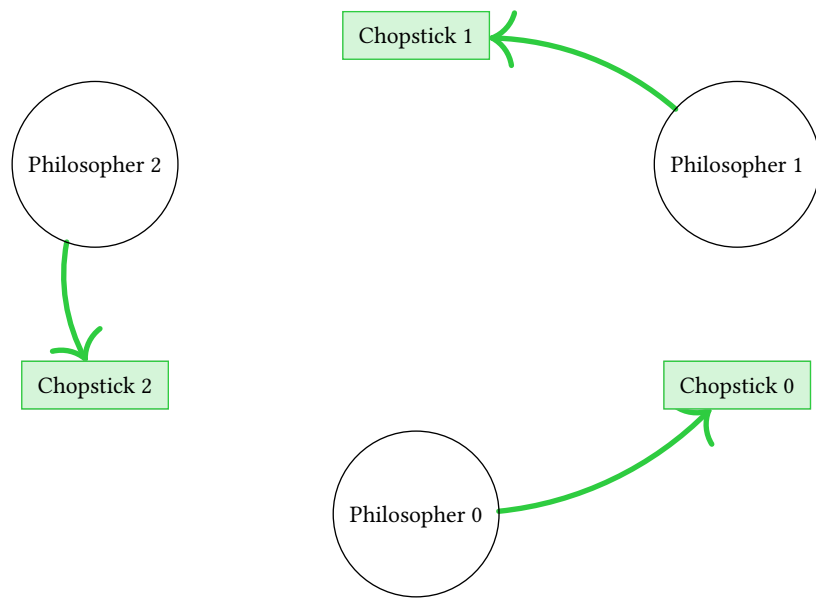


5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

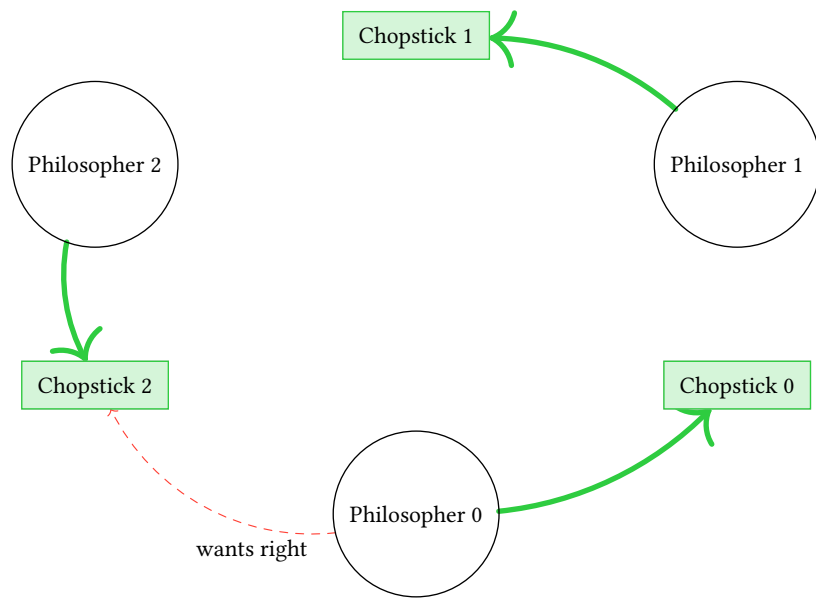


5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

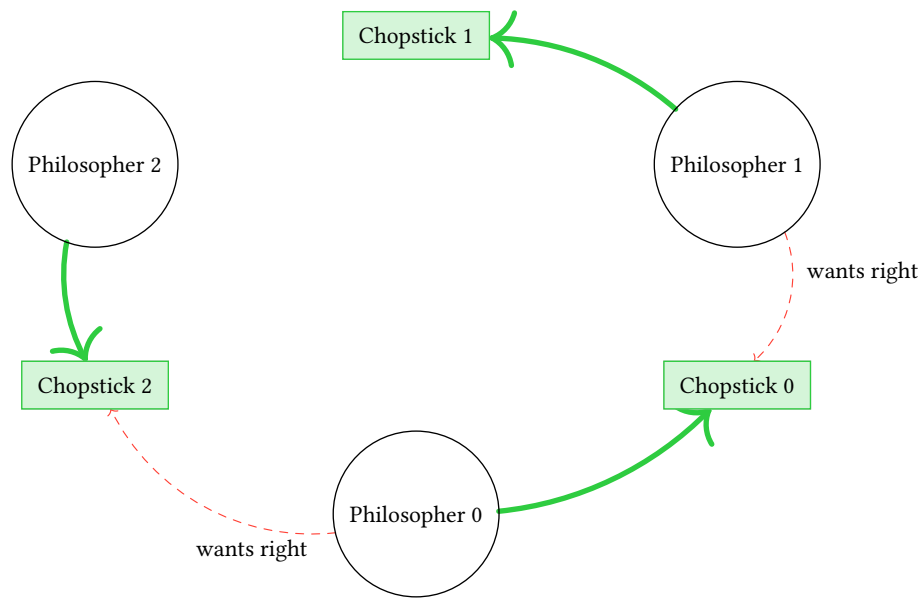


5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

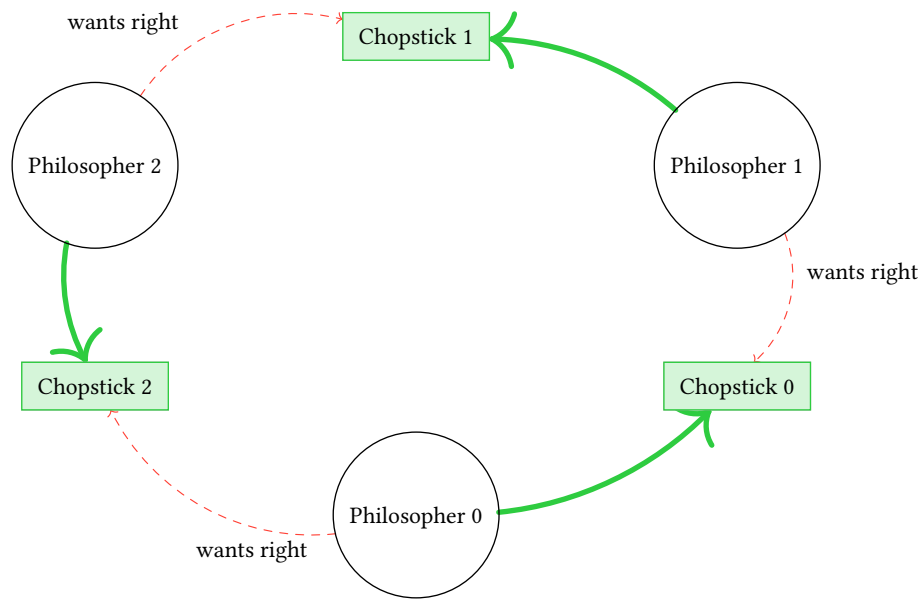


5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

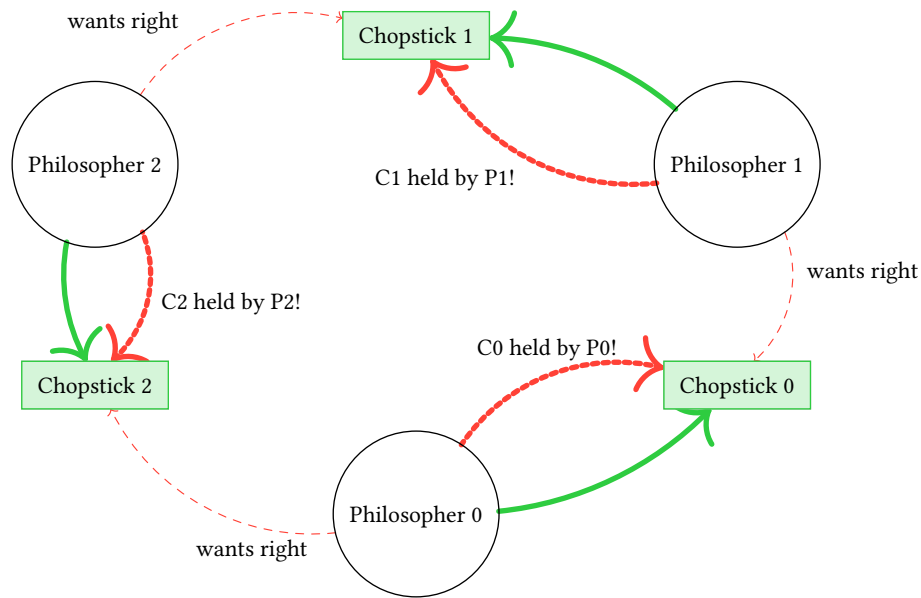


5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

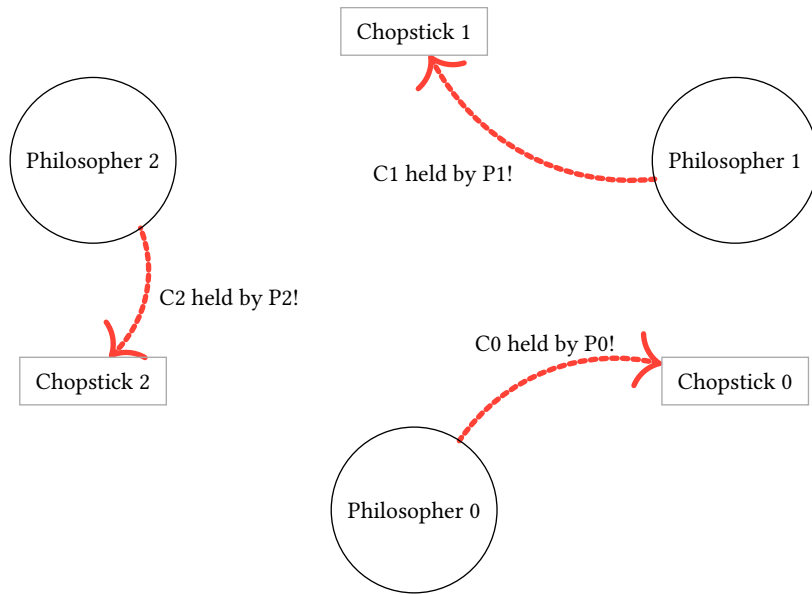


5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

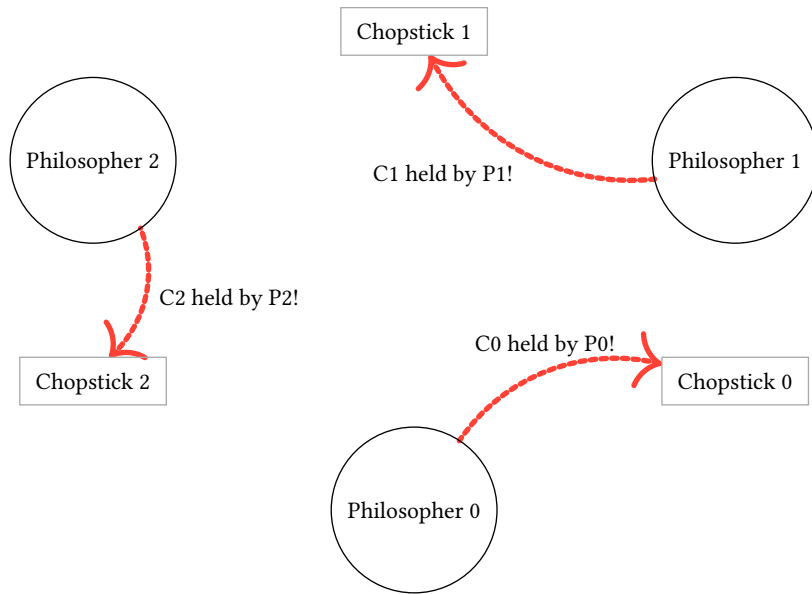
- Trying to grab
- Holding successfully
- - - Wants but blocked
- Blocking reason

5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- - - Wants but blocked
- • • • Blocking reason

Deadlock! All four Coffman conditions met:

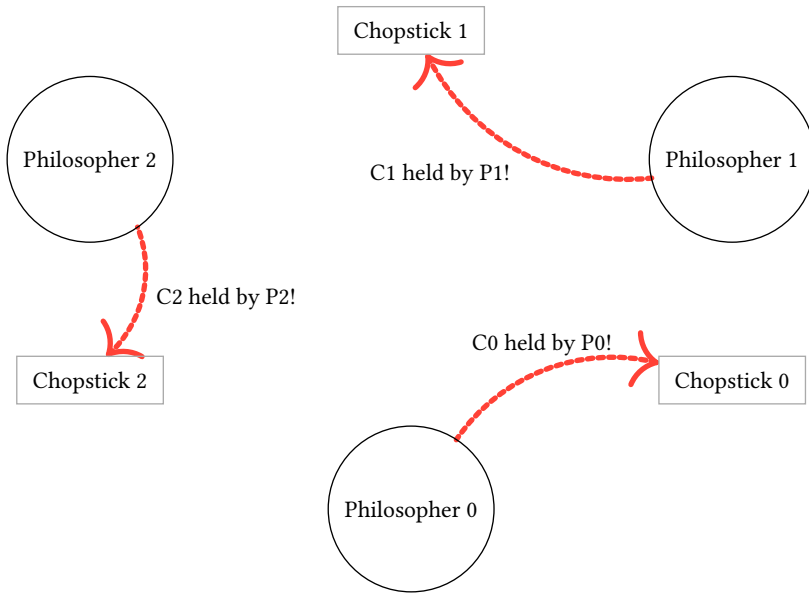
- Mutual exclusion: each chopstick held by exactly one philosopher
- Hold and wait: each philosopher holds left chopstick, waits for right
- No preemption: philosophers cannot take chopsticks from each other
- Circular wait: P0 needs C2 (held by P2) → P2 needs C1 (held by P1) → P1 needs C0 (held by P0)

5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- - - Wants but blocked
- • • • Blocking reason

Deadlock! All four Coffman conditions met:

- Mutual exclusion: each chopstick held by exactly one philosopher
- Hold and wait: each philosopher holds left chopstick, waits for right
- No preemption: philosophers cannot take chopsticks from each other
- Circular wait: P0 needs C2 (held by P2) → P2 needs C1 (held by P1) → P1 needs C0 (held by P0)

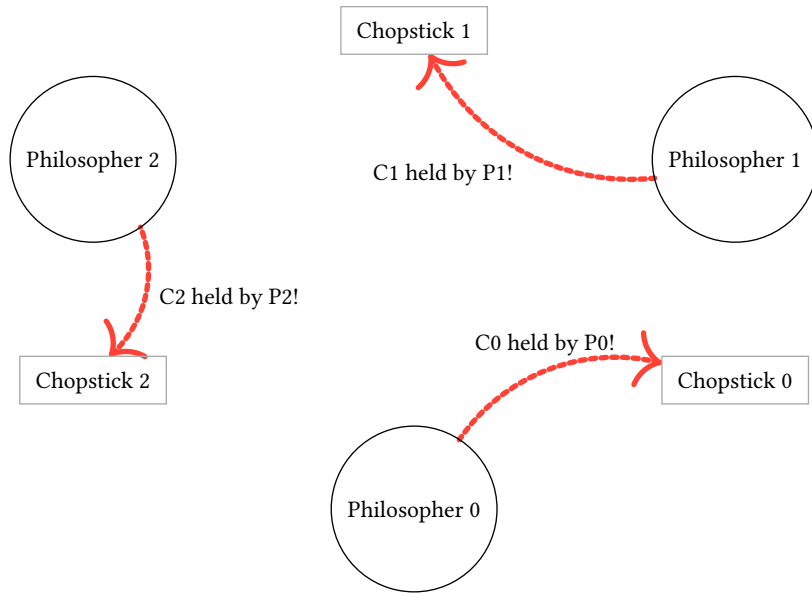
Which Coffman condition should we break to solve this?

5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- - - Wants but blocked
- • • • Blocking reason

Deadlock! All four Coffman conditions met:

- Mutual exclusion: each chopstick held by exactly one philosopher
- Hold and wait: each philosopher holds left chopstick, waits for right
- No preemption: philosophers cannot take chopsticks from each other
- Circular wait: P0 needs C2 (held by P2) → P2 needs C1 (held by P1) → P1 needs C0 (held by P0)

Which Coffman condition should we break to solve this?

Circular wait is easiest: change the lock acquisition order for some philosophers.

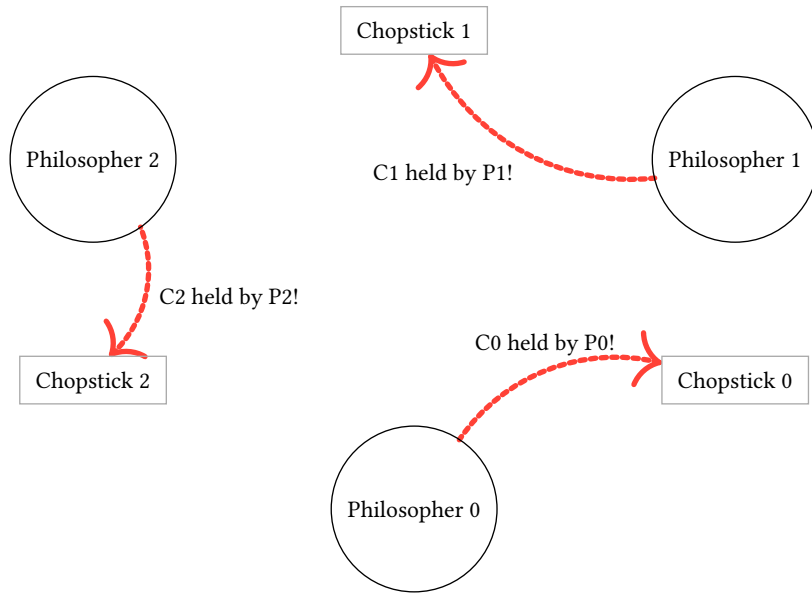
How can we break the symmetry?

5.5. Dining Philosophers deadlock

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- - - Wants but blocked
- ... Blocking reason

Deadlock! All four Coffman conditions met:

- Mutual exclusion: each chopstick held by exactly one philosopher
- Hold and wait: each philosopher holds left chopstick, waits for right
- No preemption: philosophers cannot take chopsticks from each other
- Circular wait: P0 needs C2 (held by P2) → P2 needs C1 (held by P1) → P1 needs C0 (held by P0)

Which Coffman condition should we break to solve this?

Circular wait is easiest: change the lock acquisition order for some philosophers.

How can we break the symmetry?

Make at least one philosopher acquire chopsticks in a different order than the others.

5.5. Dining Philosophers deadlock

Exercise:

- `examples/philosophers.rs`

Hints:

- All philosophers currently pick up left chopstick first, then right
- Breaking the circular pattern requires asymmetry
- You only need to change the behavior for one or more philosophers

Solutions are provided as `*-solution.rs` files.

5.6. Best practices for Mutexes

Structure code to minimize shared mutable state in mutexes.

Alternatives:

- Use `RwLock<T>` for many readers, few writers
- Use `Atomic*` types for simple counters/flags

If really necessary:

- Use channels for **one-directional flow** (but they are also hard to maintain)

One more exercise:

- `examples/link.rs` (you need to have `openssl` installed, use the `flake.nix` or your package manager)

Solutions are provided as `*-solution.rs` files.